

Theoretical Questions

Question 1.1.a

Two lazy lists are equal if the **head** of both lazy-lists (the first values in the lists) are equal, and if the tail function returns the same value for the head, at **every** execution, meaning at every “future index” of the lazy list, the head values will always be the same OR if they are both empty.

Question 1.1.b

We will first note that for any number *num*, the below mathematical result is the same:

$$(\text{num} * \text{num}) \bmod 2 = (\text{num} \bmod 2) * (\text{num} \bmod 2)$$

And for every even number *n*, $n * n$ is also even.

Let's prove by induction that both functions are equal, based on the definition above.

Base – first head value:

The head of *integer-from 0* is 0.

even-square-1 will run the predicate (is number even) on $0 * 0$, it will return TRUE, so it will run (sorry for this style of indentation, this is what is more comfortable for us to read):

```
(cons-lzl
  (head lzl)
  (lambda ()
    (lzl-filter p (tail lzl))
  )
)
```

Which will create a lazy-list with the squared head 0, and the lazy-filter with the even predicate on the tail. So, head is 0.

even-square-2 will create a lazy-list and the value of the head will be the square of the head of the filtered even lazy-list, since 0 is even it will pass the predicate, and the head will be 0.

So the head of the returned lazy-list is 0.

Both are equal.

Step - let's assume this is true for n , meaning the n th head values of the returned lazy-list are equal.

We will prove it is true for the $n+1$ as well, meaning the $(n+1)$ th head values of the returned lazy-list are equal.

The n th value is **even**, since it passed the predicate of **even-square-1**, and let us denote it as n :

Now, in **even-square-1** since n is the n th value of the returned lazy-list, this means $\sqrt{n}$ was even.

After n passed the predicate and became the head of lazy-list, the next value from *integers-from 0* is odd and it will **not** pass the predicate (as $\sqrt{n} + 1$ is odd, and odd*odd is still odd), and the returned value will be the lazy filter with the even predicate on the tail of the squared lazy list.

Now $\sqrt{n} + 2$ is even, so the squared value of it will also be even, it will pass the predicate, and it will be the head, meaning the lazy-list will be:

$[0, \dots, n, n + 2 * \sqrt{n} + 1, \text{tail}]$, that is our $(n+1)$ th value from **even-square-1**.

In **even-square-2**, n is even is the n th value so before calling the square function it was $\sqrt{n}$.

Since n is even, so is $\sqrt{n}$, which is how it passed the filtering when ``head lzl`` was executed.

So the next number is $\sqrt{n} + 1$, which again we know is odd. So when ``tail lzl`` is executed this value will not pass the predicate as it is not even.

Then the predicate will run on the next value which is $\sqrt{n} + 2$, which is even, and it will pass.

Then the square function will run on it, which will result in $n + 2 * \sqrt{n} + 1$, so our lazy-list will look like this:

$[0, \dots, n, n + 2 * \sqrt{n} + 1, \text{tail}]$, that is our $(n+1)$ th value from **even-square-2**.

The $(n+1)$ th values are equal in both lazy-lists, we proved the induction.

Question 2.a

Lets say F is a function and $F\$$ is the Success-Fail Continuation version.

They are equal only if:

$F: T1 \rightarrow T2 \mid T_fail$

And $F\$$ is defined as:

F: $T1 * [T2 \rightarrow T3] * [() \rightarrow T3]$$

And for every input of type $T1$ lets call it X and any functions for success or failure, if F succeeded and returned value v , meaning $F(X) = v$ then F(X,success,failure)$ must return $success(v)$.

If F fails on the input X , meaning $F(X) = \text{fail}$ where fail is a value of T_fail (like ``fail`), then F(X,success,failure)$ must return $failure()$.

Question 2.b

We will prove they are equal by looking at every scenario of the association list.

1. List is empty

In both functions the condition of “empty?” will return true.

- a. `get-value` will return `fail`.
- b. `get-value$` will return the value of the failure continuation function `fail`.

2. List is not empty, and in the first pair in the list, the key is the key we are looking for.

Both functions will pass the condition `eq?`

- a. `get-value` will return the value of the pair.
- b. `get-value$` will return the value of the success continuation function `success` and the input for `success` will be the value of the pair.

3. List is not empty, and in the first pair in the list, the key is `not` the key we are looking for.

Both functions will get to the “else” condition.

- a. `get-value` will return the result of `get-value` with the input being:
 - i. the rest of the list (`cdr`)
 - ii. the same key as the current input.
- b. `get-value` will return the result of `get-value` with the input being:
 - i. the rest of the list (`cdr`)
 - ii. the same key as the current input.

- iii. The same success continuation function as the current input.
- iv. The same failure continuation function as the current input.

As we can see, in all situations we get the required behavior that we defined in 2.a.

In case 3 both functions will continue on the rest of list (cdr), but will eventually both stop on case 1 or 2 (assuming the list is of finite length).

Question 3

Note: We wrote equations inside these brackets [], and substitutions inside these brackets {}.

Question 3.1.1

1. Start: $[t(s(s), G, s, p, t(K), s) = t(s(G), G, s, p, t(K), U)], \{ \}$
 - a. Both are sides of the equations are complicated expressions with the same structure -> break to multiple smaller equations.
2. $[s(s) = s(G), G = G, s = s, p = p, t(K) = T(K), s = U], \{ \}$
 - a. Remove constants or variables that are equal to each other.
3. $[s(s) = s(G), s = U], \{ \}$
 - a. Take equation “ $s(s) = s(G)$ ”, do the same as in step 1.
4. $[s = U], \{s = G\}$
 - a. Take equation “ $s = U$ ”, add to the substitution.
5. $[], \{s = U, s = G\}$

Question 3.1.2

1. Start: $[g(l, M, g, G, U, g, v(M)) = g(l, v(U), g, v(M), v(G), g, v(M))], \{ \}$
 - a. Both are sides of the equations are complicated expressions with the same structure -> break to multiple smaller equations.
2. $[l = l, M = v(U), g = g, G = v(M), U = v(G), g = g, v(M) = v(M)], \{ \}$
 - a. Remove constants or variables that are equal to each other.
3. $[M = v(U), G = v(M), U = v(G)], \{ \}$
 - a. Take first equation $M = v(U)$
4. $[G = v(M), U = v(G)], \{M = v(U)\}$
 - a. Take first equation $G = v(M)$
 - b. $\{M = v(U)\} \circ \{G = v(M)\} = \{M = v(U), G = v(v(U))\}$

5. $[U = v(G)], \{M = v(U), G = v(v(U))\}$
 - a. $\{M = v(U), G = v(v(U))\} \circ \{U = v(G)\} = \{M = v(v(G)), G = v(v(v(G)))\}$
 - b. **ERROR**, this causes failure since G is defined by a “function” with G inside of it.

Question 3.1.3

1. Start: $[m(M,N) = n(M,N)], \{\}$
 - a. While both sides have the same amount of arguments and the variables are the same, but m and n are different predicts, so this **fails**.

Question 3.1.4

1. Start: $[p([v | [V | VV]]) = p([[v | V] | VV])], \{\}$
 - a. Both side have one argument and the same predict -> break to a smaller equation
2. $[v | [V | VV] = [v | V] | VV], \{\}$
 - a. Both represent <first item in list, rest of items in list>, so we will compare first=first' and rest=rest'.
3. $[v = [v | V], [V | VV] = VV], \{\}$
 - a. **ERROR**, we get that VV are is defined by itself in the list, which is not allowed. So this results in **failure**.

Question 3.1.5

1. Start: $[g([T]) = g(T)], \{\}$
 - a. Same predict, one item -> break to smaller equation
2. $[[T] = T], \{\}$
 - a. **ERROR**, T is defined by list of T, which is not allowed, so this results in **failure**.

Question 3.3.b

We found at least one successful computation path (we actually found infinite amount), therefore it is a **success** proof tree.

We found both infinite success paths and infinite failure paths, meaning it is an **infinite** proof tree.

Question 3.3.c

As we showed in the tree, there exists a successful path, meaning there are legal substitutions that satisfy the rules. So **yes**, the query is provable from the given program.

Question 3.3.d

When you add cons to the language you add the ability to construct inductively nested data structures, meaning the data structures are allowed to have unbounded size, which makes the language **undecidable**.